Développement Web Full Stack et DevOps

TP5 - Travail collaboratif avec Git et GitHub

Travail	En binôme ou trinôme
Environnement	Git, GitHub, HTML, CSS, JavaScript

1 Objectif du TP

Dans ce TP, l'objectif principal n'est pas de développer une application complexe.

Le but est d'apprendre à **travailler à plusieurs sur le même projet** avec Git et GitHub.

À la fin du TP, chaque groupe doit avoir pratiqué :

- un dépôt GitHub commun ;
- l'ajout de collaborateurs ;
- l'utilisation des branches `main` et `dev` ;
- la création de branches `feature/...` ;
- les commandes `git add`, `git commit`, `git push` et `git pull` ;
- la création de Pull Requests ;
- la fusion de plusieurs branches ;
- la création et la résolution d'un conflit Git simple.

Le principe général est :

`branche personnelle` ⟶ `Pull Request` ⟶ `dev` ⟶ `Pull Request` ⟶ `main`

2 Choix du projet

Chaque groupe choisit librement le thème de son projet.

Le projet doit rester simple et permettre à chaque membre de développer une petite fonctionnalité identifiable.

Il peut s'agir de n'importe quelle application frontend adaptée au niveau du groupe.

Le projet peut contenir, par exemple :

```
projet/
|
|-- index.html
|-- style.css
‘-- app.js
```

Vous pouvez également utiliser une organisation différente si elle reste claire et cohérente.

Le but de la séance n'est pas de construire une application complexe.

Le point principal évalué est la manière dont les membres du groupe travaillent ensemble avec Git et GitHub.

3 Création du dépôt commun

Un membre du groupe crée un dépôt GitHub public et ajoute les autres membres comme collaborateurs.

Le groupe utilise :

- `main` : version stable du projet ;
- `dev` : branche d'intégration des travaux.

Chaque membre récupère le projet :

```
git clone URL_DU_DEPOT
cd tp5-collaboration
```

Puis :

```
git checkout dev
git pull
```

4 Répartition du travail

Le groupe choisit lui-même les fonctionnalités à développer.

Chaque membre doit être responsable d'une fonctionnalité différente et identifiable.

Exemples de noms de branches :

`feature/nom-fonctionnalite-1`
`feature/nom-fonctionnalite-2`
`feature/nom-fonctionnalite-3`

Pour un binôme, le projet doit comporter au minimum deux fonctionnalités différentes.

Pour un trinôme, le projet doit comporter au minimum trois fonctionnalités différentes.

Chaque étudiant crée sa branche depuis `dev`.

Exemple :

```
git checkout dev
git pull
git checkout -b feature/nom-fonctionnalite
```

Le nom de la branche doit décrire clairement la fonctionnalité réalisée.

5 Travail sur une branche

Chaque membre développe sa fonctionnalité sur sa branche.

Pendant le travail :

```
git status
```

Puis :

```
git add .
git commit -m "Add feature name"
git push -u origin feature/nom-fonctionnalite
```

Les messages de commit doivent être clairs.

Exemples :

```
Add first feature
Add second feature
Improve interface
Fix display issue
```

Évitez :

```
test
update
final
final2
```

6 Pull Request vers dev

Une fois la fonctionnalité terminée et envoyée sur GitHub, l'étudiant crée une Pull Request :

`feature/nom-fonctionnalite` ⟶ `dev`

Avant de fusionner :

- vérifier les fichiers modifiés ;
- vérifier les commits ;
- vérifier que la fonctionnalité fonctionne ;
- vérifier que le travail d'un autre membre n'a pas été supprimé.

Après la fusion, les autres membres récupèrent la nouvelle version :

```
git checkout dev
git pull
```

7 Exercice obligatoire : créer un conflit

Le groupe doit provoquer volontairement un petit conflit Git.

Le conflit doit rester simple.

Deux membres créent chacun une nouvelle branche depuis la même version de `dev`.

Par exemple :

```
feature/change-a
feature/change-b
```

Les deux étudiants doivent modifier volontairement la même ligne ou la même petite zone du même fichier.

Le fichier peut être `index.html`, `style.css`, `app.js` ou un autre fichier du projet.

Par exemple, deux membres peuvent modifier différemment :

- un titre ;
- un texte ;
- une variable ;
- une fonction ;
- une même partie de l'interface.

Chaque étudiant enregistre ensuite sa modification :

```
git add .
git commit -m "Update shared section"
git push
```

Une première branche est fusionnée dans `dev`.

Lors de l'intégration de la deuxième branche, Git peut signaler un conflit.

8 Résoudre le conflit

Git peut afficher des marqueurs comme :

```
<<<<<<< HEAD
version presente dans dev
=======
version de votre branche
>>>>>>> feature/change-b
```

Cela signifie que Git trouve deux modifications différentes au même endroit et ne sait pas laquelle conserver automatiquement.

Le groupe doit analyser les deux versions et choisir le résultat final.

Supprimez ensuite les marqueurs du conflit :

```
<<<<<<<
=======
>>>>>>>
```

Puis enregistrez la résolution :

```
git add .
git commit -m "Resolve merge conflict"
git push
```

Chaque membre doit comprendre pourquoi le conflit est apparu.

9 Fusion finale vers main

Lorsque toutes les fonctionnalités fonctionnent dans `dev`, le groupe crée une dernière Pull Request :

$$\texttt{dev} \longrightarrow \texttt{main}$$

Avant la fusion finale, vérifiez :

- la recherche ;
- le filtre ;
- le compteur si le groupe est composé de trois membres ;
- l'absence de conflit restant ;
- l'historique des commits ;
- les différentes branches ;
- les Pull Requests réalisées.

10 Résultat attendu

À la fin de la séance, le dépôt GitHub doit montrer :

- la branche `main` ;
- la branche `dev` ;
- les branches de travail ;
- des commits réalisés par les différents membres ;
- les Pull Requests ;
- les merges ;
- la résolution d'au moins un conflit ;
- la version finale fonctionnelle du projet.

Chaque étudiant doit avoir une contribution identifiable dans l'historique Git.

11 Vérification en classe

Chaque membre peut être amené à expliquer :

- sur quelle branche il travaille ;
- ce qu'il a modifié ;
- pourquoi il a créé cette branche ;
- ce que font `git add`, `git commit` et `git push` ;
- ce qu'est une Pull Request ;
- pourquoi un conflit est apparu ;
- comment le groupe l'a résolu.

12 À retenir

Une fonctionnalité ⟶ une branche ⟶ des commits ⟶ push ⟶ Pull Request ⟶ dev

dev stable ⟶ Pull Request ⟶ main

Le but de ce TP est de comprendre qu'un projet collaboratif ne consiste pas seulement à partager des fichiers.

Chaque membre travaille sur sa branche, enregistre son travail, partage ses modifications et les intègre progressivement dans le projet commun.